

Deadline. The homework is due **11:59pm, Mon, 04/20/2026**. You must submit your solutions (in pdf format generated by LaTeX) via GradeScope.

Late Policy. You have up to four grace days (calendar day) for the entire quarter. You don't lose any point by using grace days. If you decide to use grace days, please specify how many grace days you would like to use and the reason at the beginning of your submission.

Academic Integrity. You can discuss with your classmates or use online resources. In particular,

- **Collaboration Policy.** You can get help from the instructor, TAs, or your classmates, but only after you have thought about the problems on your own. **If you discussed with anyone, you need to acknowledge them in your report.**
- **Other sources.** It is OK to get inspiration (but not solutions) from books or online resources, again after you have carefully thought about the problems on your own. **If you use any reference or webpage, or discussed with anyone, you must cite it fully and completely in your report.**
- **AI Tools.** If you use ChatGPT or similar AI tools, you have to attach the full conversation to make it clear what help you obtained from them.

However, you **CANNOT**:

- share your report/code with anyone else,
- read other's report/code,
- copy anything from other source,
- get help from others without acknowledging them in your report, or use any relevant resources (e.g., online article/AI tools such as ChatGPT) without citing them.

Otherwise it will be considered cheating. We reserve the right to deduct points for using such material beyond reason. **In summary, you must write up your solution independently, and type your answers word by word on your own: close the book/notes/online resources when you are writing your final answers. Add all the citations and collaborators' names to the end.**

Write-up. Please use LaTeX to prepare your solutions. For all problems, **please explain how you get the answer** instead of directly giving the final answer, except for some special cases (will be specified in the problem). For all algorithm design problems, please describe using **natural language**. You can present pseudocode if you think that helps to illustrate your idea, but please do not only give some code without explanation. **In grading, we will reward not only correctness but also clarity and simplicity. To avoid losing points for hard-to-understand solutions, you should make sure your solutions are either self-explanatory or contains good explanations.** See some more details here <https://www.cs.ucr.edu/~yihans/teaching/214/S26/assignments/>

Bonus questions. Bonus questions are not required, and the points are not counted in the basic 100 points of the class. Most of the bonus questions are just slightly harder than the regular questions, but some can be extremely hard. In a nutshell, most of them are worthy trying, but don't be frustrated if you cannot solve *all* of them :) We will not provide too many hints/solutions for bonus problems, but feel free to discuss them with your classmates.

1 Work and Span (3pts)

Compute the work and span of the following algorithms. Explain briefly.

For all of them, you can assume `do_something` takes constant cost. Use $\Theta(\cdot)$ to present your answer.

(1) (1pt)

```
void compute(int n) {
    parallel_for i = 0 to n {
        parallel_for i = 0 to n {
            do_something();
        } } }
```

(2) (1pt)

```
void compute(int n) {
    if (n == 0) return;
    parallel_do {
        compute(n/8);
        compute(n/8);
    }
    do_something();
}
```

(3) (1pt)

```
void compute(int n) {
    if (n == 0) return;
    parallel_do {
        compute(n/2);
        compute(n/2);
    }
    parallel_for i = 0 to sqrt(n) {
        do_something();
    }
}
```

(4) (bonus, 0.5pt + 1 candy)

```
void compute(int n) {
    if (n < 4) return;
    parallel_for i = 0 to sqrt(n) { compute(sqrt(n)); }
    parallel_for i = 0 to n {
        do_something();
    }
}
```

(5) (bonus, 0.5pt + 1 candy)

```
void compute(int n) {
    if (n < 4) return;
    parallel_for i = 0 to sqrt(n) { do_something(); }
    compute(n - sqrt(n));
}
```

For the last two bonus problems, please supplement the details for solving the recurrences if needed. The bonus candy is awarded only if you fully solve the problem.

2 (Another) Parallel Scan Algorithm (5pts + 1 pt + 1 candy)

In this problem, we'll see another parallel algorithm for computing prefix sum. Assume the input array is $A[1..n]$ and the output (prefix sum) array is $B[1..n]$.

1. (1pt) In class, we've learned two scan algorithms, using divide-and-conquer and decrease-and-conquer, respectively. What are the work and span recurrences and bounds for them, respectively?

In practice, we've already seen that we need to do granularity control when the problem size is small. Here's another way to do this "manually". In particular, we can split the problem size into k chunks, each with n/k elements, and try to process the chunks in parallel. Inside each chunk, the elements will be processed sequentially. The high-level idea of the algorithm works as follows:

- Step 1. Split the array into k chunks, each of size about $\lceil n/k \rceil$ except for probably the last one.
- Step 2. Create k parallel tasks, each of them is to compute the sum of all elements inside each chunk. Assume the sums are stored in an array $S[1..k]$.
- Step 3. Using $S[1..k]$, compute $B[n/k]$, $B[2n/k]$, $B[3n/k]$, ..., $B[n]$ sequentially.
- Step 4. Create k parallel tasks, each of them in charge of a chunk. Each chunk will be processed sequentially, and all chunks are parallelized. For each chunk, we will compute the values in $B[]$ based on array A , and the results Step 3.
2. (1pts) How do you compute the values in Step 3.?
 3. (1pts) How do you compute the values in Step 4.?
 4. (0.5pts) What is the total work of this algorithm?
 5. (0.5pts) What is the span of this algorithm? You can use the parameter k in your bound.
 6. (1pts) What is the best span bound you could get for this algorithm? To get the best span bound, what parameter k should you choose?
 7. (Bonus, 1 pt and 1 candy) What happens if we use $k = \Theta(n)$? In this case, each chunk size will be a constant. You will likely find out that this will give you low parallelism - the span of the algorithm will be huge. Can you slightly modify this algorithm to make it have better span? Does this new algorithm look like an algorithm we learned in class?

Note: This algorithm actually achieves very good performance in practice (with reasonable value for k) — usually similar performance to the two algorithms we've learned in class. That's because the number of processors we have today is still much smaller than the problem size, and thus splitting the problem size into reasonably small chunks is good enough.